ΛProof Adapter Coordination Interface Specification

Version: 1.0.0-draft

Status: Public Specification

Overview

The Adapter Coordination Interface (ACI) defines how external tools—web browsers, email clients, payment processors, healthcare systems, or any computational agent—integrate with the ΛProof authorization layer. Adapters are the enforcement boundary between **intent** (what an agent wants to do) and **action** (what the tool actually executes).[1]

This specification ensures:

- Tools only execute actions that have passed policy evaluation
- Capability tokens bind actions to verified intents
- Audit events are emitted for every enforcement decision
- The system fails closed—no token, no action

Core Principles

1. Fail-Closed Enforcement

Adapters must block all actions unless a valid capability token is presented. If CSL checks fail or drift exceeds bounds, the adapter enters silent mode—action blocked, receipt archived only.[2]

2. Zero-Knowledge of Policy Logic

Adapters receive decisions (ALLOW/DENY) and proof requirements from the policy engine but never see how risk classification or CSL commutation works. The adapter's job is *enforcement*, not *evaluation*.[1]

3. Tool-Agnostic Interface

The same interface works for web navigation, email transmission, financial transfers, or device emissions. Tool-specific semantics are encapsulated in adapter implementations.[2]

4. Audit-First Architecture

Every enforcement decision generates an Archivum receipt containing only commitments and metadata—no PII.[1]

Interface Definitions

Adapter Base Interface

```
interface LambdaProofAdapter {
  // Adapter identity
  readonly adapterId: string;
  readonly toolName: string;
  readonly version: string;

  // Lifecycle
  initialize(config: AdapterConfig): Promise<void>;
  shutdown(): Promise<void>;

  // Core enforcement
  enforceAction(
    token: CapabilityToken,
```

```
    action: ToolAction
  ): Promise<EnforcementResult>;

  // Token validation
  validateToken(token: CapabilityToken): Promise<TokenValidation>;

  // Audit emission
  emitAuditEvent(event: AdapterAuditEvent): Promise<string>;

  // Health & status
  getStatus(): AdapterStatus;
}
```

Configuration

```
interface AdapterConfig {
  // Policy engine connection
  policyEndpoint: string;

  // Archivum connection for receipts
  archivumEndpoint: string;

  // Cryptographic material
  adapterSigningKey: string;  // For signing audit events
  tokenVerificationKey: string;  // Public key for token verification

  // Operational parameters
  enforcementMode: 'strict' | 'permissive' | 'audit-only';
  silentModeEnabled: boolean;
  maxDriftThreshold: number;  // Default: 0.3

  // Tool-specific configuration
  toolConfig: Record<string, unknown>;
}
```

Capability Token Format

```
interface CapabilityToken {
  // Token identity
```

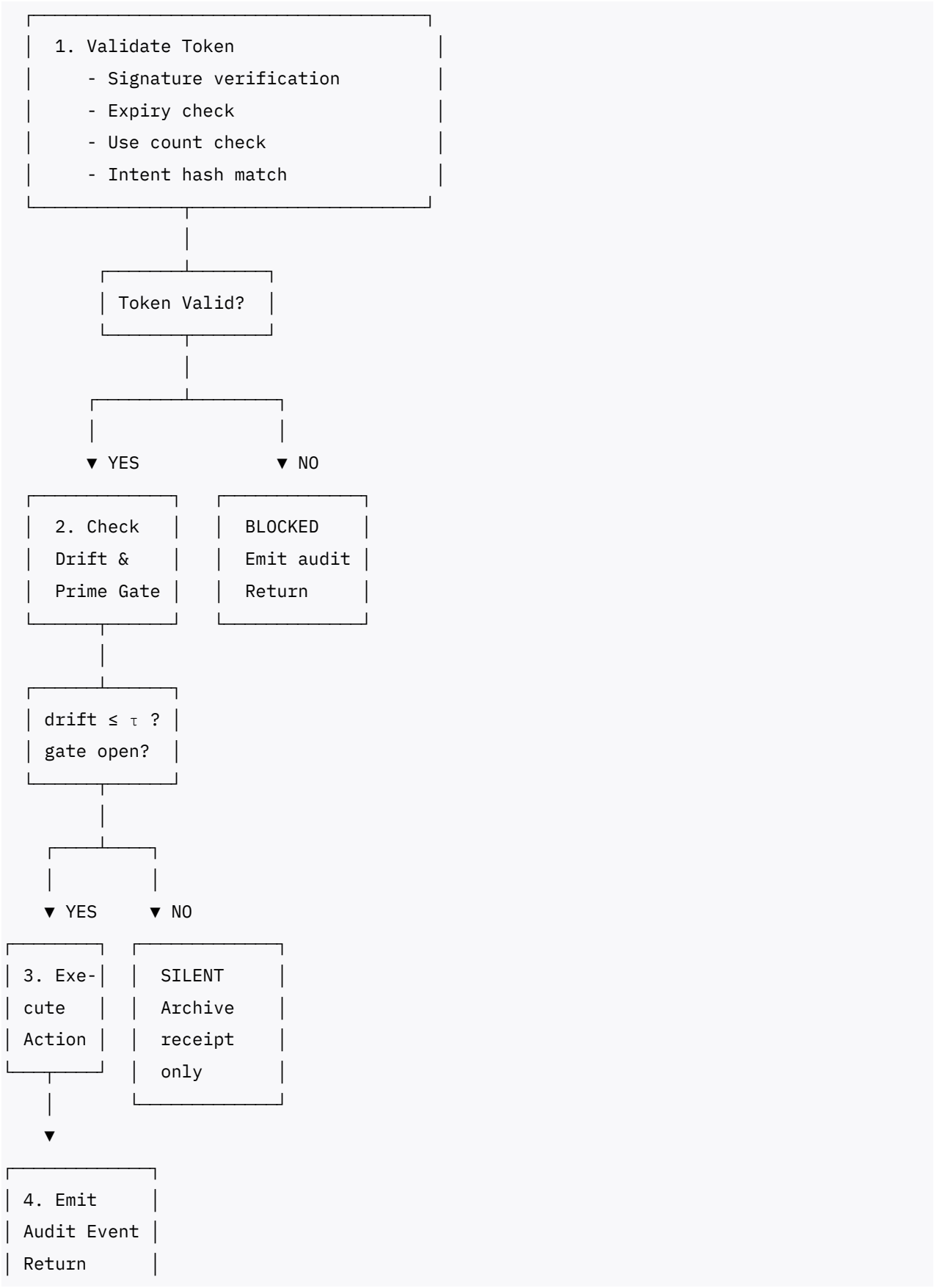
```
  tokenId: string;
  intentHash: string;  // SHA256 of canonicalized intent

  // Authorization scope
  subject: {
    subjectId: string;
    sessionId?: string;
  };
  tool: string;
  action: string;
  params: Record<string, unknown>;

  // Constraints
  constraints: {
    notBefore: number;  // Unix timestamp
    notAfter: number;   // Unix timestamp
    maxUses: number;
    usedCount: number;
  };

  // Proof level required
  proofLevel: 'NONE' | 'ATTESTATION' | 'ZK';

  // Cryptographic binding
  signature: string;  // EdDSA signature over token body
  issuer: string;     // Token issuer public key

  // Audit chain
  primeIndex: number;       // Prime epoch at issuance
  archivumCid?: string;     // Content-addressed receipt reference
}
```

Enforcement Result

```
interface EnforcementResult {
  status: 'EXECUTED' | 'BLOCKED' | 'SILENT';

  // Execution details (if EXECUTED)
  executionId?: string;
```

```
  toolResponse?: unknown;

  // Blocking reason (if BLOCKED or SILENT)
  reason?: BlockReason;

  // Audit reference
  auditEventId: string;
  archivumCid: string;

  // Timing
  enforcementTimestamp: number;
  executionDuration?: number;
}

type BlockReason =
  | 'TOKEN_INVALID'
  | 'TOKEN_EXPIRED'
  | 'TOKEN_USED'
  | 'DRIFT_EXCEEDED'
  | 'PRIME_GATE_CLOSED'
  | 'CSL_VIOLATION'
  | 'TOOL_ERROR';
```

Enforcement Flow


1. Validate Token
- Signature verification
- Expiry check
- Use count check
- Intent hash match
Token Valid?
YES
NO
2. Check Drift & Prime Gate
BLOCKED
Emit audit
Return
drift ≤ τ ?
gate open?
YES
NO
3. Execute Action
SILENT
Archive receipt only
4. Emit Audit Event
Return

```
| EXECUTED    |
└─────────────┘
```

Audit Event Schema

Every enforcement decision produces an audit event:

```
interface AdapterAuditEvent {
  // Event identity
  eventId: string;
  eventType: 'ENFORCEMENT';

  // Context
  adapterId: string;
  tokenId: string;
  intentHash: string;

  // Decision
  decision: 'EXECUTED' | 'BLOCKED' | 'SILENT';
  reason?: BlockReason;

  // Timing
  timestamp: number;
  primeIndex: number;

  // Drift metrics (no PII)
  driftScore?: number;
  driftThreshold: number;

  // Tool response summary (no PII)
  executionSuccess?: boolean;
  executionDuration?: number;

  // Cryptographic binding
  previousEventHash: string;  // Chain linkage
  eventHash: string;          // Poseidon(event fields)
  signature: string;          // Adapter signature
```

```
}
```

Privacy Guarantee: Audit events contain only hashed commitments and operational metadata. No user data, intent parameters, or tool responses are included unless explicitly consented.[1]

Adapter Lifecycle

Initialization

```
async function initializeAdapter(config: AdapterConfig): Promise<void> {
  // 1. Verify cryptographic material
  await verifySigningKey(config.adapterSigningKey);
  await verifyVerificationKey(config.tokenVerificationKey);

  // 2. Connect to policy engine
  await connectPolicyEngine(config.policyEndpoint);

  // 3. Connect to Archivum
  await connectArchivum(config.archivumEndpoint);

  // 4. Load tool-specific configuration
  await loadToolConfig(config.toolConfig);

  // 5. Emit initialization event
  await emitAuditEvent({
    eventType: 'ADAPTER_INITIALIZED',
    adapterId: this.adapterId,
    timestamp: Date.now()
  });
}
```

Shutdown

```
async function shutdownAdapter(): Promise<void> {
  // 1. Emit shutdown event
  await emitAuditEvent({
    eventType: 'ADAPTER_SHUTDOWN',
```

```
    adapterId: this.adapterId,
    timestamp: Date.now()
  });

  // 2. Flush pending audit events
  await flushAuditBuffer();

  // 3. Close connections
  await disconnectPolicyEngine();
  await disconnectArchivum();
}
```

Reference Implementations

Web Navigation Adapter

```
class WebNavigationAdapter implements LambdaProofAdapter {
  readonly adapterId = 'web-navigation-v1';
  readonly toolName = 'browser';
  readonly version = '1.0.0';

  async enforceAction(
    token: CapabilityToken,
    action: ToolAction
  ): Promise<EnforcementResult> {
    // Validate token
    const validation = await this.validateToken(token);
    if (!validation.valid) {
      return this.blocked(token, validation.reason);
    }

    // Check drift and prime gate
    const drift = await this.computeDrift(token);
    if (drift > this.config.maxDriftThreshold) {
      return this.silent(token, 'DRIFT_EXCEEDED');
    }
```

```
    // Execute navigation
    const url = action.params.url as string;
    const response = await this.navigate(url);

    // Emit audit event
    const auditId = await this.emitAuditEvent({
      eventType: 'ENFORCEMENT',
      decision: 'EXECUTED',
      tokenId: token.tokenId,
      executionSuccess: response.ok
    });

    return {
      status: 'EXECUTED',
      executionId: response.requestId,
      auditEventId: auditId
    };
  }

  private async navigate(url: string): Promise<NavigationResponse> {
    // Tool-specific implementation
    return browser.navigate(url);
  }
}
```

Payment Transfer Adapter

```
class PaymentTransferAdapter implements LambdaProofAdapter {
  readonly adapterId = 'payment-transfer-v1';
  readonly toolName = 'payments';
  readonly version = '1.0.0';

  async enforceAction(
    token: CapabilityToken,
    action: ToolAction
  ): Promise<EnforcementResult> {
    // Higher scrutiny for financial actions
    if (token.proofLevel !== 'ZK') {
      return this.blocked(token, 'INSUFFICIENT_PROOF_LEVEL');
```

```
    }

    // Validate token with full verification
    const validation = await this.validateToken(token);
    if (!validation.valid) {
      return this.blocked(token, validation.reason);
    }

    // Verify ZK proof if required
    if (token.proofLevel === 'ZK') {
      const zkValid = await this.verifyZKProof(token.zkProof);
      if (!zkValid) {
        return this.blocked(token, 'ZK_PROOF_INVALID');
      }
    }

    // Execute transfer
    const transfer = await this.executeTransfer({
      amount: action.params.amount,
      recipient: action.params.recipient,
      memo: action.params.memo
    });

    // Emit audit event
    const auditId = await this.emitAuditEvent({
      eventType: 'ENFORCEMENT',
      decision: 'EXECUTED',
      tokenId: token.tokenId,
      executionSuccess: transfer.success
    });

    return {
      status: 'EXECUTED',
      executionId: transfer.transactionId,
      auditEventId: auditId
    };
  }
}
```

Error Codes

Code	Name	Description
`ACI-001`	`TOKEN_SIGNATURE_INVALID`	Token signature verification failed
`ACI-002`	`TOKEN_EXPIRED`	Token notAfter timestamp exceeded
`ACI-003`	`TOKEN_NOT_YET_VALID`	Token notBefore timestamp not reached
`ACI-004`	`TOKEN_USES_EXHAUSTED`	Token maxUses exceeded
`ACI-005`	`INTENT_HASH_MISMATCH`	Action params don't match intent hash
`ACI-006`	`DRIFT_EXCEEDED`	Drift score above threshold
`ACI-007`	`PRIME_GATE_CLOSED`	Current prime epoch gate is closed
`ACI-008`	`CSL_VIOLATION`	CSL commutation check failed
`ACI-009`	`TOOL_UNAVAILABLE`	Underlying tool not responding
`ACI-010`	`ADAPTER_NOT_INITIALIZED`	Adapter not properly initialized

Security Considerations

Token Replay Prevention

Tokens include a monotonic `usedCount` that adapters must track. Single-use tokens (`maxUses: 1`) are invalidated immediately after first enforcement.[3]

Time Binding

Tokens have strict `notBefore` and `notAfter` timestamps. Adapters must reject tokens outside this window regardless of signature validity.

Prime Gate Synchronization

Adapters must synchronize with the current prime epoch. Actions submitted during a closed gate enter silent mode—archived but not executed.[2]

Audit Chain Integrity

Each audit event includes `previousEventHash`, creating a verifiable chain. Tampering with any event invalidates the entire chain.

Conformance Requirements

Adapters claiming ΛProof conformance **MUST**:

1. Implement the full `LambdaProofAdapter` interface
2. Validate token signatures before any enforcement decision
3. Check drift against configured threshold
4. Emit audit events for all enforcement decisions
5. Enter silent mode when drift exceeds bounds or prime gate is closed
6. Never execute actions without valid tokens (fail-closed)
7. Never log or transmit PII in audit events

Adapters **MAY**:

- Implement tool-specific extensions to the base interface
- Add additional validation steps before enforcement
- Support multiple tool actions within a single adapter

Adapters **MUST NOT**:

- Execute actions when tokens are invalid, expired, or exhausted
- Expose policy evaluation logic or risk classification details
- Store raw intent parameters or user data beyond hashed commitments

---

Versioning

This specification follows semantic versioning. Breaking changes to the interface require a major version increment. Adapters should declare their supported ACI version in the `version` field.

References

- ΛProof Interface Blueprint[1]
- Ξ-Constitution: Prime-Lawful Arithmetic Consciousness[2]
- MTPI RootContract Specifications[3]

Document Status: Draft for public review
Feedback: [Submit issues to the ΛProof repository]

This document establishes prior art for the Adapter Coordination Interface pattern. It is published as a defensive publication under CC0 to prevent third-party enclosure while enabling permissionless implementation.

⁂

---

1. LProof-Interface-Blueprint.pdf
2. LProof.pdf
3. MTPI-RootContract.pdf
4. 2.ΛRootContract II & III - MTPI - NODE.js.pdf
    a. Λproof Monorepo — Plain-language Overview V0.pdf
    b. Λ-trace Overview.pdf
    c. Λproof Banking Protocols.pdf
    d. Λproof Healthcare Protocols V0.1.pdf
    e. Λdapps On Web4 - Developer Overview.pdf
5. Building a ΛDapp_ A Developer's Implementation Guide.pdf
6. MTPI_Instructions.pdf

7. MTPI_RootContract_Project.pdf
8. Patent Search Results.pdf
9. The ΛProof Architecture_ A Comprehensive Analysis of its Implications for Web4.pdf
10. The ΛProof Architecture_ Web4.pdf
11. Verifiable, Privacy-Preserving Applications.pdf
12. Conscious_Sovereignty_Layer.pdf
13. Milestone-Vision-Timeframe.csv
14. Lets further develop the SAFE_ awesome—let's turn.pdf
15. ΞConstitution.pdf
16. Λᴾ-Archivum.pdf
    f. Λproof × Mtpi Lawful-by-design Integration — Implementation Plan.pdf